

Project Summary Report

Github: https://github.com/CasperL1218/cs4782_graph_attention_network

Introduction:

Many real-world datasets naturally form graphs, such as citation networks, protein interaction networks, and social networks. Traditional CNNs work well on grid-structured data like images, but they cannot directly handle graph structures because graphs have irregular connections and varying neighborhood sizes.

Earlier graph neural networks had important limitations: GCNs depend on graph spectral methods and generalize poorly to unseen graphs, while GraphSAGE uses fixed neighborhood sampling that can lose structural information.

This project reproduces Graph Attention Networks (GAT) from Petar Veličković et al. (ICLR 2018). GAT uses self-attention to learn different importance weights for neighboring nodes during aggregation.

Key advantages of GAT: Efficient: avoids expensive eigendecomposition. Expressive: learns attention weights instead of uniform averaging. Inductive: generalizes to unseen graphs using node features

Chosen Result:

We reproduced the transductive node classification results on the Cora and Citeseer datasets from the original GAT paper. The paper reports $83.0 \pm 0.7\%$ accuracy on Cora and $72.5 \pm 0.7\%$ on Citeseer, outperforming GCN by small but important margins.

Table 2: Summary of results in terms of classification accuracies, for Cora, Citeseer and Pubmed. GCN-64* corresponds to the best GCN result computing 64 hidden features (using ReLU or ELU).

<i>Transductive</i>			
Method	Cora	Citeseer	Pubmed
MLP	55.1%	46.5%	71.4%
ManiReg (Belkin et al., 2006)	59.5%	60.1%	70.7%
SemiEmb (Weston et al., 2012)	59.0%	59.6%	71.7%
LP (Zhu et al., 2003)	68.0%	45.3%	63.0%
DeepWalk (Perozzi et al., 2014)	67.2%	43.2%	65.3%
ICA (Lu & Getoor, 2003)	75.1%	69.1%	73.9%
Planetoid (Yang et al., 2016)	75.7%	64.7%	77.2%
Chebyshev (Defferrard et al., 2016)	81.2%	69.8%	74.4%
GCN (Kipf & Welling, 2017)	81.5%	70.3%	79.0%
MoNet (Monti et al., 2016)	$81.7 \pm 0.5\%$	—	$78.8 \pm 0.3\%$
GCN-64*	$81.4 \pm 0.5\%$	$70.9 \pm 0.5\%$	$79.0 \pm 0.3\%$
GAT (ours)	$83.0 \pm 0.7\%$	$72.5 \pm 0.7\%$	$79.0 \pm 0.3\%$

We chose this experiment because the fixed dataset splits make comparison straightforward and because the small improvement over GCN requires correctly implementing the attention mechanism itself. We focused on Cora and Citeseer instead of Pubmed since Pubmed requires additional architectural tuning that was outside the scope of this project.

Methodology:

We implemented a two-layer GAT in PyTorch from scratch without using GATConv. The first layer uses 8 attention heads with 8-dimensional outputs, concatenated into a 64-dimensional representation with ELU activation and dropout ($p=0.6$). The second layer uses one attention head with log-softmax activation.

Attention scores were computed as: $e_{ij} = \text{LeakyReLU}(a_{src}^T W h_i + a_{dst}^T W h_j)$, instead of explicitly concatenating $[W h_i || W h_j]$, reducing memory usage. Self-loops were added during each forward pass, and we implemented `return_attn=True` to visualize learned attention weights.

Dataset and Evaluation: We evaluated the model on the standard Planetoid benchmarks: Cora: 2,708 nodes, 10,556 edges, 7 classes and Citeseer: 3,327 nodes, 9,104 edges, 6 classes. Both datasets used the standard transductive split: 20 labeled nodes per class for training, 500 validation nodes, 1000 test nodes. Performance was measured using mean test accuracy and standard deviation across 100 runs.

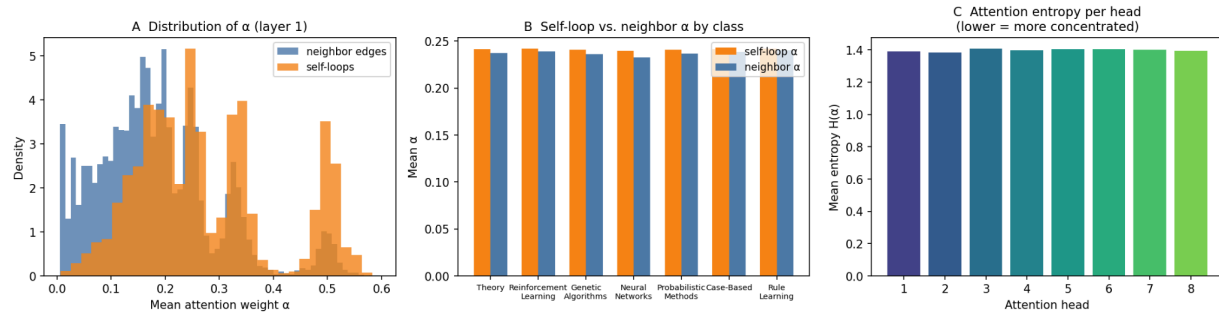
Training Modification: We matched the paper’s main hyperparameters: Adam optimizer, learning rate = 0.005, weight decay = 5×10^{-4} , dropout = 0.6. We also modified early stopping by resetting patience when either validation loss or validation accuracy improved, preventing premature stopping when accuracy was still increasing.

Result & Analysis:

Our re-implementation closely reproduces the paper's results, with a small accuracy gap attributable to framework differences.

Method	Cora	Citeseer
GCN (Kipf & Welling, 2017)	81.5%	70.3%
GAT — paper (Veličković et al., 2018)	83.0 ± 0.7%	72.5 ± 0.7%
GAT — ours	80.9 ± 1.0%	69.0 ± 1.2%

Several factors likely contributed to this difference: a) the original paper used TensorFlow while we used PyTorch, which differs slightly in initialization and dropout behavior, b) we did not fix random seeds across runs, increasing variance, c) preprocessing differences, particularly in Citeseer edge construction.



Our attention visualization showed mostly uniform weights, meaning GAT on average behaved close to a GCN rather than strongly differentiating neighbors. Attention heads showed little specialization, and self-loops were slightly stronger than neighbor weights, indicating reliance on node features over graph structure.

This explains why our results are closer to GCN than the paper's GAT performance. With limited labeled data, the model struggled to learn meaningful attention patterns. Overall, our results support that attention improves graph learning, but performance is sensitive to data size, initialization, and implementation details.

Reflection:

Lesson learned: Small implementation details significantly affect reproduction results. Differences between TensorFlow and PyTorch (especially initialization and dropout) likely contributed to performance gaps, and small datasets like Cora are highly sensitive to randomness due to limited labels.

Attention visualization was especially useful, revealing near-uniform attention, limited head specialization, and reliance on self-loops, which explained the GCN-like behavior. We also found that simple design choices (split attention, dual early stopping, visualization tools) greatly improved debugging and stability.

What We Would Do Differently: given more time, we would: use fixed random seeds to reduce variance, reproduce the PPI inductive benchmark, run attention analysis on larger datasets to test whether head specialization emerges with more training data.

Future work could explore: a) GATv2, which improves the expressiveness of the original attention mechanism b) incorporating edge features for heterogeneous and knowledge graphs c) extending GAT to graph-level prediction using attention-based pooling methods

Reference:

- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph Attention Networks. ICLR 2018.
- Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR 2017.
- Sen, P., et al. (2008). Collective Classification in Network Data. AI Magazine. (Cora & Citeseer datasets)
- Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR Workshop 2019.
- Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.